

مشروع مادة البرمجة 2

نظام إدارة مطعم داخل فندق

البرنامج الدراسي:

دبلوم في العلوم السياحية

الاختصاص:

إدارة سياحية وتطبيقات تقانة معلومات

المرحلة:

السنة الأولى — الفصل الثاني

اسم الطالبة:

مروة الصحني

جدول المحتويات | Table of Contents

المقدمة	
فكرة عمل البرنامج	
الصفوف المستخدمة	
MenuItem — الصف المجرد	
Meal — صف الوجبة	
Drink — صف المشروب	
Employee — صف الموظف	
Table — صف الطاولة	
Order — صف الطلب	
Main — الصف الرئيسي	
الكود المصدري الكامل	
مثال على تشغيل البرنامج	

المقدمة | Introduction

في هذا المشروع قمت بإنشاء برنامج بلغة Java يهدف إلى إدارة مطعم داخل فندق. يسمح البرنامج بإدخال بيانات موظف وطاولة، ثم إضافة وجبات ومشروبات إلى طلب الزبون، وعرض تفاصيل الطلب كاملة مع الأسعار النهائية. الهدف من المشروع هو تطبيق المفاهيم الأساسية في البرمجة الكائنية مثل الوراثة، الصف المجرد، التحميل الزائد للبواني، إعادة تعريف التوابع، تعدد الأشكال، والإدخال من المستخدم.

فكرة عمل البرنامج | Program Overview

يعتمد البرنامج على مجموعة من الصفوف المترابطة. يقرأ الصف الرئيسي بيانات الموظف والطاولة من المستخدم، ثم ينشئ كائن طلب ويطلب من المستخدم إدخال عدد من العناصر (وجبات أو مشروبات) تضاف إلى الطلب واحدا تلو الآخر. بعد الانتهاء من الإدخال يتم عرض تفاصيل الطلب كاملة مع السعر الإجمالي.

الصفوف المستخدمة في المشروع | Classes Used

MenuItem	صف مجرد — يمثل أي عنصر في قائمة المطعم (وجبة أو مشروب)
Meal	يرث من MenuItem — يمثل وجبة مع خاصية الإضافات
Drink	يرث من MenuItem — يمثل مشروبا بثلاثة أحجام
Employee	يمثل موظفا في المطعم — يوضح التحميل الزائد للبواني
Table	يمثل طاولة في المطعم — يوضح التحميل الزائد للبواني
Order	يمثل طلب الزبون — يخزن الوجبات والمشروبات في مصفوفة MenuItem[]
Main	الصف الرئيسي — يقرأ المدخلات ويعرض تفاصيل الطلب

وصف الصفوف | Class Descriptions

[abstract] MenuItem.java

الوصف	المحدد	النوع	الاسم
اسم العنصر	protected	String	name
سعر العنصر الأساسي	protected	double	price
رقم العنصر	protected	int	itemId
الوصف		التابع	
باني يستقبل رقم العنصر واسمه وسعره		MenuItem(int, String, double)	
يعرض معلومات العنصر الأساسية		displayInfo()	
تابع مجرد — ينفذ في الصفوف الوارثة		calculateFinalPrice()	

[extends MenuItem] Meal.java

الوصف	المحدد	النوع	الاسم
هل تحتوي الوجبة على إضافات	private	boolean	hasExtras
الوصف		التابع	
باني الوجبة		Meal(int, String, double, boolean)	
يضيف 15% إذا كانت هناك إضافات		calculateFinalPrice()	
يعرض معلومات الوجبة وحالة الإضافات		displayInfo()	

[extends MenuItem] Drink.java

الوصف	المحدد	النوع	الاسم
حجم المشروب: Small / Medium / Large	private	String	size
الوصف		التابع	
باني المشروب		Drink(int, String, double, String)	
Small: ×1 Medium: ×1.10 Large: ×1.20		calculateFinalPrice()	
يعرض معلومات المشروب وحجمه		displayInfo()	

[class] Employee.java

الوصف	المحدد	النوع	الاسم
رقم الموظف	private	int	employeeId
اسم الموظف	private	String	name
الدور الوظيفي	private	String	role
الراتب	private	double	salary
الوصف		التابع	
باني كامل		Employee(int, String, String, double)	
باني مختصر — role=Waiter, salary=300		Employee(int, String)	
يعرض معلومات الموظف		displayEmployeeInfo()	

[class] Table.java			
الوصف	المحدد	النوع	الاسم
رقم الطاولة	private	int	tableNumber
عدد المقاعد	private	int	numberOfSeats
حالة الحجز	private	boolean	isReserved
الوصف		التابع	
باني كامل		Table(int, int, boolean)	
باني مختصر — isReserved=false		Table(int, int)	
يغير حالة الطاولة إلى محجوزة		reserveTable()	
يعرض معلومات الطاولة		displayTableInfo()	

[class] Order.java			
الوصف	المحدد	النوع	الاسم
رقم الطلب	private	int	orderId
كائن الطاولة	private	Table	table
كائن الموظف	private	Employee	employee
مصفوفة العناصر	private	MenuItem[]	items
عدد العناصر المضافة	private	int	itemCount
الوصف		التابع	
باني الطلب		Order(int, Table, Employee, int)	
يضيف عنصرا إلى الطلب		addItem(MenuItem)	
يحسب مجموع الأسعار النهائية		calculateTotal()	
يعرض تفاصيل الطلب كاملة		displayOrderDetails()	

المفاهيم البرمجية المطبقة | OOP Concepts Applied

الصف المجرد (Abstract Class)

تم إنشاء الصف المجرد MenuItem الذي يحتوي على التابع `calculateFinalPrice()`. لا يمكن إنشاء كائن منه مباشرة، بل يجب أن ترثه الصفوف الأخرى وتنفذ هذا التابع.

الوراثة (Inheritance)

يرث كل من Drink و Meal من الصف MenuItem باستخدام `extends`. يستخدم `super()` في بائيهما لاستدعاء بائي الصف الأب.

إعادة تعريف التوابع (Method Overriding)

يعيد كل من Drink و Meal تعريف التابعين `calculateFinalPrice()` و `displayInfo()` ليناسب طبيعة كل منهما. يشار إلى ذلك بـ `@Override`.

التحميل الزائد للبواني (Constructor Overloading)

يملك كل من Employee و Table بائيين: الأول يستقبل جميع البيانات، والثاني يستقبل بيانات محدودة مع قيم افتراضية.

تعدد الأشكال (Polymorphism)

تخزن كائنات Drink و Meal في مصفوفة من نوع `MenuItem[]` داخل صف `Order`. عند استدعاء `calculateFinalPrice()` ينفذ التابع المناسب لكل كائن تلقائياً.

محددات الوصول (Access Modifiers)

تستخدم `protected` لمتغيرات MenuItem حتى تستطيع الصفوف الوراثة الوصول إليها، و `private` لحماية بيانات Employee و Table و Order و Meal و Drink.

الإدخال من المستخدم (Scanner)

يستخدم الصف Main أداة Scanner لقراءة جميع البيانات من المستخدم بشكل تفاعلي.

الكود المصدري الكامل | Full Source Code

MenuItem.java

```
public abstract class MenuItem {

    protected String name;
    protected double price;
    protected int itemId;

    // Constructor: receives item id, name, and price
    public MenuItem(int itemId, String name, double price) {
        this.itemId = itemId;
        this.name = name;
        this.price = price;
    }

    // Display basic item information
    public void displayInfo() {
        System.out.println("Item ID: " + itemId);
        System.out.println("Name: " + name);
        System.out.println("Base Price: " + price);
    }

    // Abstract method: each subclass calculates its own final price
    public abstract double calculateFinalPrice();
}
```

Meal.java

```
public class Meal extends MenuItem {

    // Indicates whether the meal includes extras (fries, salad, extra sauce, etc.)
    private boolean hasExtras;

    // Constructor: receives meal id, name, price, and whether it has extras
    public Meal(int itemId, String name, double price, boolean hasExtras) {
        super(itemId, name, price);
        this.hasExtras = hasExtras;
    }

    // If meal has extras, add 15% to the base price; otherwise price stays the same
    @Override
    public double calculateFinalPrice() {
        if (hasExtras) {
            return price * 1.15;
        }
        return price;
    }

    // Display meal info and whether it has extras
    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Has Extras: " + (hasExtras ? "Yes" : "No"));
    }
}
```

```

        System.out.printf("Final Price: %.2f%n", calculateFinalPrice());
    }
}

```

Drink.java

```

public class Drink extends MenuItem {

    // Size of the drink: Small, Medium, or Large
    private String size;

    // Constructor: receives drink id, name, price, and size
    public Drink(int itemId, String name, double price, String size) {
        super(itemId, name, price);
        this.size = size;
    }

    // Small: same price | Medium: +10% | Large: +20%
    @Override
    public double calculateFinalPrice() {
        if (size.equalsIgnoreCase("Medium")) {
            return price * 1.10;
        } else if (size.equalsIgnoreCase("Large")) {
            return price * 1.20;
        }
        return price; // Small
    }

    // Display drink info including its size
    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Size: " + size);
        System.out.printf("Final Price: %.2f%n", calculateFinalPrice());
    }
}

```

Employee.java

```

public class Employee {

    private int employeeId;
    private String name;
    private String role;
    private double salary;

    // Constructor Overloading – full constructor with all employee data
    public Employee(int employeeId, String name, String role, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.role = role;
        this.salary = salary;
    }

    // Constructor Overloading – minimal constructor: role defaults to waiter, salary to 300
    public Employee(int employeeId, String name) {
        this.employeeId = employeeId;
    }
}

```

```

        this.name = name;
        this.role = "Waiter";
        this.salary = 300;
    }

    // Display all employee information
    public void displayEmployeeInfo() {
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Name: " + name);
        System.out.println("Role: " + role);
        System.out.println("Salary: " + salary);
    }

    public String getName() {
        return name;
    }
}

```

Table.java

```

public class Table {

    private int tableNumber;
    private int numberOfSeats;
    private boolean isReserved;

    // Constructor Overloading – full constructor with reservation status
    public Table(int tableNumber, int numberOfSeats, boolean isReserved) {
        this.tableNumber = tableNumber;
        this.numberOfSeats = numberOfSeats;
        this.isReserved = isReserved;
    }

    // Constructor Overloading – table is not reserved by default
    public Table(int tableNumber, int numberOfSeats) {
        this.tableNumber = tableNumber;
        this.numberOfSeats = numberOfSeats;
        this.isReserved = false;
    }

    // Mark this table as reserved
    public void reserveTable() {
        isReserved = true;
    }

    // Display table information
    public void displayTableInfo() {
        System.out.println("Table Number: " + tableNumber);
        System.out.println("Number of Seats: " + numberOfSeats);
        System.out.println("Reserved: " + (isReserved ? "Yes" : "No"));
    }

    public int getTableNumber() {
        return tableNumber;
    }
}

```

Order.java

```

public class Order {

    private int orderId;
    private Table table;
    private Employee employee;
    private MenuItem[] items; // Meals and Drinks stored as MenuItem (polymorphism)
    private int itemCount;

    // Constructor: receives order id, table, employee, and max number of items
    public Order(int orderId, Table table, Employee employee, int maxItems) {
        this.orderId = orderId;
        this.table = table;
        this.employee = employee;
        this.items = new MenuItem[maxItems];
        this.itemCount = 0;
    }

    // Add a menu item (Meal or Drink) to the order
    public void addItem(MenuItem item) {
        if (itemCount < items.length) {
            items[itemCount] = item;
            itemCount++;
        }
    }

    // Sum the final prices of all items in the order
    public double calculateTotal() {
        double total = 0;
        for (int i = 0; i < itemCount; i++) {
            total += items[i].calculateFinalPrice();
        }
        return total;
    }

    // Display order id, table info, employee name, all items, and total price
    public void displayOrderDetails() {
        System.out.println("\nOrder Details:");
        System.out.println("Order ID: " + orderId);
        System.out.println("Table Number: " + table.getTableNumber());
        System.out.println("Employee: " + employee.getName());
        System.out.println("\nItems:");
        for (int i = 0; i < itemCount; i++) {
            System.out.printf("%s - Final Price: %.2f\n",
                              items[i].name, items[i].calculateFinalPrice());
        }
        System.out.printf("Total Price: %.2f\n", calculateTotal());
    }
}

```

Main.java

```

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}

```

```
// --- Input employee data ---
System.out.print("Enter employee id: ");
int employeeId = scanner.nextInt();
scanner.nextLine();

System.out.print("Enter employee name: ");
String employeeName = scanner.nextLine();

System.out.print("Enter employee role: ");
String employeeRole = scanner.nextLine();

System.out.print("Enter employee salary: ");
double employeeSalary = scanner.nextDouble();

Employee employee = new Employee(employeeId, employeeName, employeeRole, employeeSalary);

// --- Input table data ---
System.out.print("Enter table number: ");
int tableNumber = scanner.nextInt();

System.out.print("Enter number of seats: ");
int numberOfSeats = scanner.nextInt();

Table table = new Table(tableNumber, numberOfSeats);
table.reserveTable();

// --- Input number of items ---
System.out.print("Enter number of items in order: ");
int numItems = scanner.nextInt();
scanner.nextLine();

Order order = new Order(1, table, employee, numItems);

// --- Input each item ---
for (int i = 1; i <= numItems; i++) {
    System.out.println("\nItem " + i + ":");
    System.out.print("Enter item type, 1 for Meal, 2 for Drink: ");
    int itemType = scanner.nextInt();
    scanner.nextLine();

    if (itemType == 1) {
        System.out.print("Enter meal id: ");
        int mealId = scanner.nextInt();
        scanner.nextLine();

        System.out.print("Enter meal name: ");
        String mealName = scanner.nextLine();

        System.out.print("Enter meal price: ");
        double mealPrice = scanner.nextDouble();
        scanner.nextLine();

        System.out.print("Does the meal have extras? ");
        boolean hasExtras = scanner.nextBoolean();
        scanner.nextLine();
    }
}
```

```
        order.addItem(new Meal(mealId, mealName, mealPrice, hasExtras));

    } else {
        System.out.print("Enter drink id: ");
        int drinkId = scanner.nextInt();
        scanner.nextLine();

        System.out.print("Enter drink name: ");
        String drinkName = scanner.nextLine();

        System.out.print("Enter drink price: ");
        double drinkPrice = scanner.nextDouble();
        scanner.nextLine();

        System.out.print("Enter size: ");
        String size = scanner.nextLine();

        order.addItem(new Drink(drinkId, drinkName, drinkPrice, size));
    }
}

// --- Display complete order details ---
order.displayOrderDetails();

scanner.close();
}
```

مثال على تشغيل البرنامج | Sample Program Run

عند تشغيل البرنامج يقوم المستخدم بإدخال بيانات الموظف والطاولة، ثم اختيار عدد العناصر ونوع كل منها. فيما يلي مثال كامل على الإدخال والإخراج:

```
Enter employee id: 1
Enter employee name: Ahmad
Enter employee role: Waiter
Enter employee salary: 400

Enter table number: 5
Enter number of seats: 4

Enter number of items in order: 2

Item 1:
Enter item type, 1 for Meal, 2 for Drink: 1
Enter meal id: 101
Enter meal name: Burger
Enter meal price: 5.0
Does the meal have extras? true

Item 2:
Enter item type, 1 for Meal, 2 for Drink: 2
Enter drink id: 201
Enter drink name: Cola
Enter drink price: 1.5
Enter size: Large

Order Details:
Order ID: 1
Table Number: 5
Employee: Ahmad

Items:
Burger - Final Price: 5.75
Cola - Final Price: 1.80
Total Price: 7.55
```

شرح حساب الأسعار النهائية | Price Calculation Explained

العنصر	ي س اس أ ل ر ع س ل ا	ة د ع ا ق ل ا	ي ئ ا ه ن ل ا ر ع س ل ا
Burger (وجبة مع إضافات)	5.00	× 1.15 (+15%)	5.75

Cola (مشروب Large)	1.50	× 1.20 (+20%)	1.80
الإجمالي			7.55